

module2_lab2

September 14, 2026

Module 2 Lab 2: Textbook Problem 4.2

Name: Ramisha Tasfia

Course: Introduction to Data Science with Python

GitHub Repository: <https://github.com/ramishatasfia5156-commits/cs82a-portfolio>

Problem 4.2: Multiplication the Long Way

Task: Write a function that multiplies two whole numbers *without* using the `*` operator. Accomplish this by adding one number to itself repeatedly inside a loop. Implement it **twice**: once with a **for** loop and once with a **while** loop, then confirm both produce the same result as Python's built-in multiplication.

Part 1: Using a for Loop

```
[1]: def multiply_for(num1, num2):  
    """Multiply num1 by num2 using a for loop (repeated addition)."""  
    result = 0  
    for i in range(num2):  
        result += num1  
    return result
```

```
# Test calls  
print("--- for loop version ---")  
print(f"7 x 6 = {multiply_for(7, 6)}")  
print(f"3 x 8 = {multiply_for(3, 8)}")  
print(f"12 x 5 = {multiply_for(12, 5)}")  
print(f"1 x 9 = {multiply_for(1, 9)}")  
print(f"0 x 4 = {multiply_for(0, 4)}")
```

```
--- for loop version ---  
7 x 6 = 42  
3 x 8 = 24  
12 x 5 = 60  
1 x 9 = 9  
0 x 4 = 0
```

Part 2: Using a while Loop

```
[2]: def multiply_while(num1, num2):  
    """Multiply num1 by num2 using a while loop (repeated addition)."""  
    result = 0  
    counter = 0  
    while counter < num2:  
        result += num1  
        counter += 1  
    return result  
  
    # Test calls  
    print("--- while loop version ---")  
    print(f"7 x 6 = {multiply_while(7, 6)}")  
    print(f"3 x 8 = {multiply_while(3, 8)}")  
    print(f"12 x 5 = {multiply_while(12, 5)}")  
    print(f"1 x 9 = {multiply_while(1, 9)}")  
    print(f"0 x 4 = {multiply_while(0, 4)}")
```

```
--- while loop version ---  
7 x 6 = 42  
3 x 8 = 24  
12 x 5 = 60  
1 x 9 = 9  
0 x 4 = 0
```

How the while Loop Terminates

The `while` loop terminates because of the `counter` variable: it starts at 0 and increments by 1 on every iteration (`counter += 1`). The loop's condition is `counter < num2`, so once `counter` reaches `num2` the condition becomes `False` and the loop exits. Since `counter` is guaranteed to increase by exactly 1 each pass, it will always reach `num2` in a finite number of steps — preventing an infinite loop.

Part 3: Verification — Both Functions Agree with *

```
[3]: test_cases = [(7, 6), (3, 8), (12, 5), (1, 9), (0, 4)]  
  
print(f"{'num1':>5} {'num2':>5} {'for result':>12} {'while result':>14} {'*_  
↪result':>10} {'match?':>8}")  
print("-" * 60)  
  
all_correct = True  
for num1, num2 in test_cases:  
    for_result = multiply_for(num1, num2)  
    while_result = multiply_while(num1, num2)  
    star_result = num1 * num2  
    match = (for_result == while_result == star_result)
```

```

    if not match:
        all_correct = False
    print(f"{num1:>5} {num2:>5} {for_result:>12} {while_result:>14}␣
↪{star_result:>10} {'YES' if match else 'NO':>8}")

print("-" * 60)
print(f"All results correct: {all_correct}")

```

num1	num2	for result	while result	* result	match?
7	6	42	42	42	YES
3	8	24	24	24	YES
12	5	60	60	60	YES
1	9	9	9	9	YES
0	4	0	0	0	YES

All results correct: True